

Guía para comenzar el Hito 3

Prácticas Profesionalizantes I

Tecnicatura Superior en Desarrollo de Software

Repositorio de ejemplo

<https://github.com/alexanderselingabc/SaludTotal2026>

Consignas

Disponibles en el campus.

Profesor: Alexander Seling

Ayudante de Cátedra: Jorge Marinoni

¿Cómo empezar?

Pueden pensar el desarrollo del Hito 3 como una continuación de lo que trabajamos en las clases anteriores.

En las últimas clases trabajamos sobre:

- la manipulación del DOM.
- el **flujo de cada usuario según su rol**;
- el **modelado de los datos del dominio en formato JSON**;

A continuación vamos a repasar estos conceptos y ver cómo **integrarlos para construir un frontend dinámico**, aplicando lo trabajado y demostrado en la clase grabada.

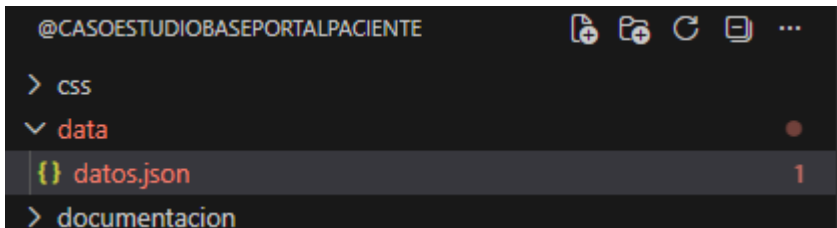
1. Modelar los datos

En las clases anteriores identificaron información que estaba escrita directamente en sus archivos HTML.

Comiencen creando:

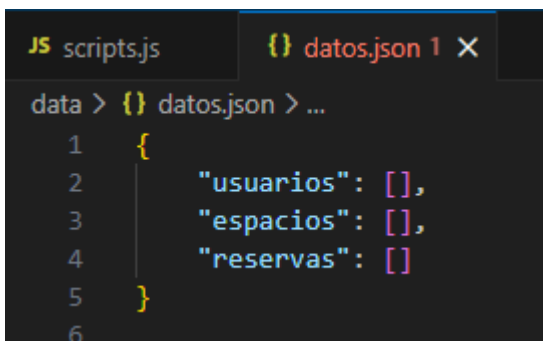
data/

└─ datos.json



y modelen allí los datos que necesita su aplicación.

Por ejemplo, si están desarrollando un sistema de reservas, podrían tener usuarios, espacios y reservas:



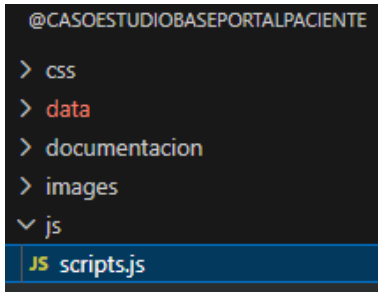
- 📝 No hace falta que el modelo sea idéntico al ejemplo. Lo importante es que represente los datos que necesita **su propio dominio**.
- ⚠️ Recuerden incluir datos suficientes para probar las funcionalidades de la aplicación.

2. Incorporar JavaScript

Creen un archivo:

js/

└─ scripts.js



y vincúlenlo desde las páginas HTML.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Solicitar Turno</title>
  <link rel="stylesheet" href="../css/styles.css">
  <script src="../js/scripts.js" defer></script>
</head>
```

A partir de este momento, parte de la información que anteriormente estaba escrita directamente en HTML deberá comenzar a generarse mediante JavaScript.

Pueden utilizar el repositorio de ejemplo para observar cómo:

- determinar qué página se está mostrando;
- construir rutas relativas;
- cargar `data/datos.json` mediante `fetch()`;
- recorrer los datos;
- generar elementos HTML dinámicamente.

Por ejemplo, en el repositorio de referencia utilizamos una función para resolver las rutas:

```
function resolverUrlRelativa(path) {
  // Resuelve la ruta tomando como referencia la raíz del proyecto.
  const dentroDePages = window.location.pathname.includes("/pages/");
  return dentroDePages
    ? `../${path}`
    : path;
}
```

De esta forma podemos escribir las rutas siempre tomando como referencia la raíz del proyecto sin tener que escribir manualmente `../` dependiendo de la página desde la que se ejecute el código:

```
const ruta = resolverUrlRelativa("data/datos.json");
```

```
window.location.href = resolverUrlRelativa("index.html");
```

```
window.location.href = resolverUrlRelativa("pages/confirmacion.html");
```

3. Cargar los datos

Una vez creado `datos.json`, prueben primero que JavaScript pueda cargarlo correctamente mediante `fetch()`.

```
const ruta = resolverUrlRelativa("data/datos.json");
const respuesta = await fetch(ruta);
const datosIniciales = await respuesta.json();
```

El método `response.json()` transforma el contenido JSON recibido en datos que JavaScript puede utilizar como objetos y arrays.

-  `async` y `await` son palabras clave de JavaScript para trabajar con operaciones asíncronas.
- `await` significa, conceptualmente: **“esperá el resultado de esta operación antes de continuar”**.
- `async` indica que una función puede utilizar `await`.

Una vez que los datos están cargados en la memoria del navegador, podemos modificar el html existente.

En el siguiente ejemplo, si JavaScript busca el elemento html de tipo “li” con id “lista-especialidades” y le agrega cada uno de las especialidades definidas en el json:

```
async function renderizarPageHome() {

  const lista = document.querySelector("#lista-especialidades");
  if (!lista) return;

  const datos = await obtenerDatosGlobales();

  for (const especialidad of datos.especialidades) {
    const article = document.createElement("article");

    article.innerHTML = `
      <h3>${especialidad.nombre}</h3>
      <p>${especialidad.descripcion}</p>
    `;

    lista.append(article);
  }
}
```

Artículos definidos en el JSON:

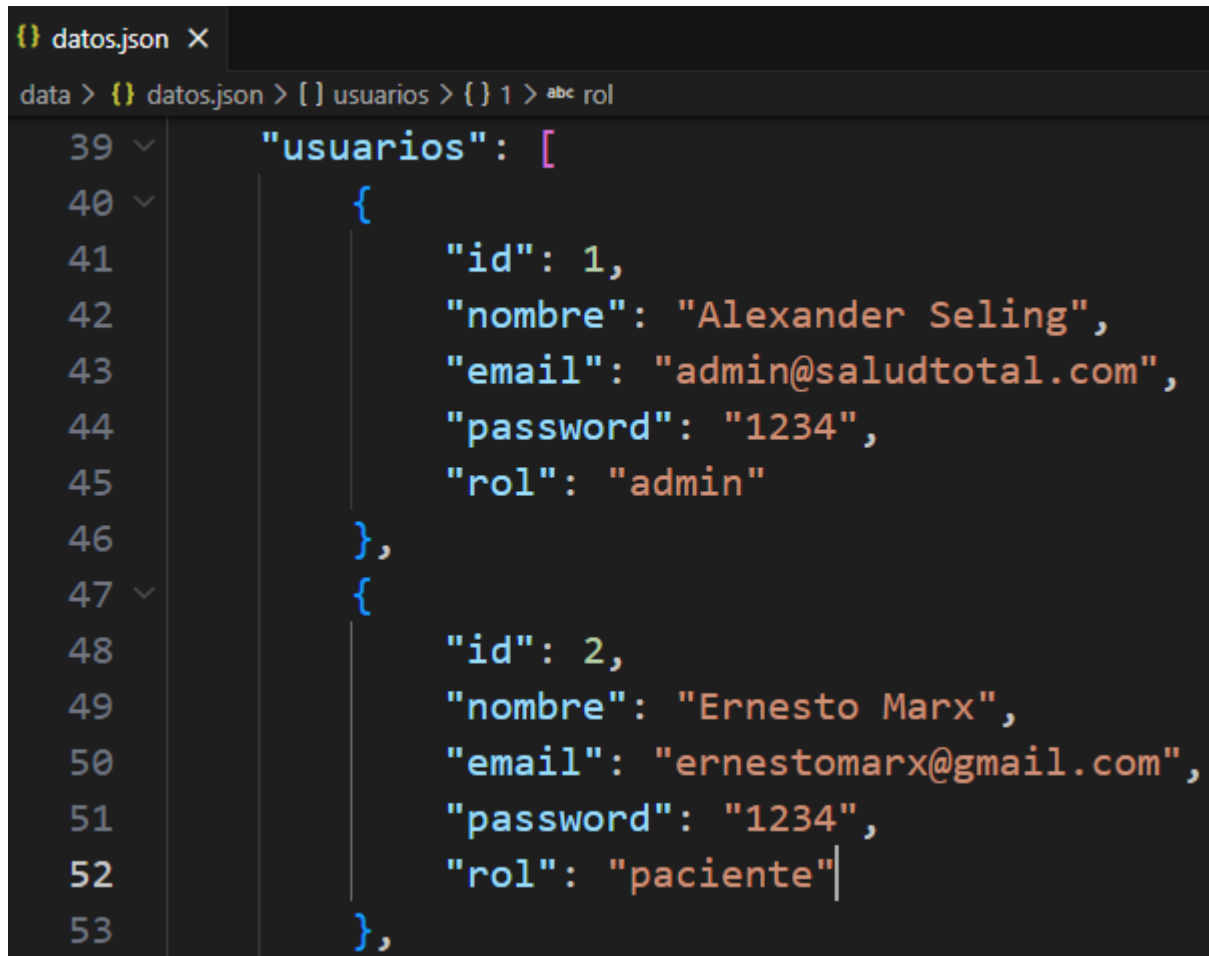
```
datos.json x
data > datos.json > ...
139 "especialidades": [
140   {
141     "nombre": "Clínica médica",
142     "descripcion": "Atención médica general, diagnóstico y se
143   },
144   {
145     "nombre": "Cardiología",
146     "descripcion": "Prevención, diagnóstico y tratamiento de
147   },
148   {
149     "nombre": "Dermatología",
150     "descripcion": "Diagnóstico y tratamiento de enfermedades
151   },
152   {
153     "nombre": "Pediatría",
154     "descripcion": "Atención integral de niños y adolescentes
155   },
156   {
157     "nombre": "Traumatología",
158     "descripcion": "Diagnóstico y tratamiento de lesiones y e
159   },
160   {
161     "nombre": "Oftalmología",
162     "descripcion": "Prevención, diagnóstico y tratamiento de
163   }
164 ]
165 ]
```

4. Crear el inicio de sesión

Modelen algunos usuarios ficticios dentro de `datos.json`.

Cada usuario deberá tener, como mínimo, los datos necesarios para iniciar sesión y determinar su rol.

Por ejemplo:



```
{} datos.json X
data > {} datos.json > [] usuarios > {} 1 > abc rol
39  ✓ "usuarios": [
40  ✓ {
41      "id": 1,
42      "nombre": "Alexander Seling",
43      "email": "admin@saludtotal.com",
44      "password": "1234",
45      "rol": "admin"
46  },
47  ✓ {
48      "id": 2,
49      "nombre": "Ernesto Marx",
50      "email": "ernestomarx@gmail.com",
51      "password": "1234",
52      "rol": "paciente"
53  },
```

Luego pueden crear una pantalla de inicio de sesión que:

1. tenga un formulario que solicite email y contraseña;
2. intercepte el envío del formulario mediante un `eventListener`;
3. utilice `event.preventDefault()` para evitar el envío tradicional del formulario;
4. busque dentro de los datos al usuario correspondiente;
5. compruebe las credenciales;
6. guarde el usuario conectado en `localStorage`.

5. Mostrar la interfaz según el rol

Una vez que puedan identificar al usuario conectado, utilicen su rol para determinar qué elementos de la interfaz debe visualizar.

Por ejemplo:

Visitante → puede ver información pública

Usuario → puede acceder a sus funcionalidades

Administrador → puede acceder a funcionalidades de administración

El objetivo es que puedan utilizar la información del usuario conectado para modificar la interfaz y controlar el acceso a determinadas páginas.

En el repositorio de ejemplo, la función `renderizarNavBar()` utiliza el rol del usuario para decidir qué elementos de la barra de navegación deben mostrarse:

```
function renderizarNavBar(usuario) {  
  
    const elementos = document.querySelectorAll(".paciente, .admin, .visitante");  
    elementos.forEach(elemento => {  
        elemento.style.display = "none";  
    });  
  
    const clase = usuario ? normalizarRol(usuario.rol) : "visitante";  
    document.querySelectorAll(`.${clase}`).forEach(elemento => {  
        elemento.style.display = "list-item";  
    });  
  
    const bienvenida = document.querySelector("#bienvenida");  
    if (bienvenida) {  
        bienvenida.textContent = usuario  
            ? `Bienvenid@ ${usuario.nombre}`  
            : "Bienvenido visitante";  
    }  
}
```

Observen que al principio ocultamos todos los elementos que tengan la clase paciente, admin o visitante.

Y luego que hayamos averiguado el rol del usuario, buscamos los elementos cuya clase coincida con ese rol (normalizado), y les aplicamos un `forEach` que les cambia el `display`.

No es necesario copiar esta función exactamente. Pueden resolverlo de otra manera según su creatividad y las características de su propio proyecto.

6. Guardar cambios en localStorage

Una vez que puedan cargar y mostrar los datos, incorporen las operaciones que correspondan a su sistema.

Por ejemplo:

- crear una reserva;
- registrar un producto;
- agregar una publicación;
- modificar un dato;
- cancelar una operación.

Cuando una acción modifique los datos, deberán guardar el nuevo estado en `localStorage`.

Para guardar objetos o arrays utilizamos `JSON.stringify()`:

```
// =====  
// LOCAL STORAGE  
// =====  
  
const STORAGE_KEY = "portalPacienteData";  
const SESSION_KEY = "portalPacienteUsuario";  
  
function guardarDatosGlobales(datos) {  
  localStorage.setItem(STORAGE_KEY, JSON.stringify(datos));  
}  
  
function guardarUsuarioActual(usuario) {  
  localStorage.setItem(SESSION_KEY, JSON.stringify(usuario));  
}
```

y para recuperarlos `JSON.parse`:

```
function obtenerUsuarioActual() {  
  const usuario = localStorage.getItem(SESSION_KEY);  
  return usuario ? JSON.parse(usuario) : null;  
}
```

En el repositorio de ejemplo utilizamos una función particular para obtener los datos globales:

```
async function obtenerDatosGlobales() {
  const ruta = resolverUrlRelativa("data/datos.json");
  const respuesta = await fetch(ruta);
  const datosIniciales = await respuesta.json();
  const datosGuardados = localStorage.getItem(STORAGE_KEY);

  if (!datosGuardados) {
    guardarDatosGlobales(datosIniciales);
    return datosIniciales;
  }

  const datosGuardadosParseados = JSON.parse(datosGuardados);

  return {
    ...datosIniciales,
    ...datosGuardadosParseados
  };
}
```

Lo que hace es combinar los datos del json con los datos del localStorage y los devuelve como si fueran la misma lista.

Esto permite que el resto del código trabaje directamente con objetos y arrays de JavaScript, sin tener que repetir constantemente la lógica de `localStorage`.

⚠ **Importante: JavaScript no modifica directamente `datos.json`.**

* El archivo JSON contiene sólo los **datos iniciales** de la aplicación.

`localStorage` permite conservar los cambios realizados durante el uso de la aplicación.

Por ejemplo, si un usuario crea una nueva reserva, el programa no modifica el archivo `datos.json`. Modifica los datos que tiene en memoria y luego guarda ese nuevo estado en `localStorage`.

7. Comprobar que los cambios persistan

Finalmente, prueben el funcionamiento completo.

Por ejemplo:

1. iniciar sesión;
2. realizar una operación;
3. comprobar que los datos cambien;
4. cambiar de página;
5. volver a la página anterior;
6. comprobar que el cambio siga presente;
7. recargar la página y volver a comprobarlo.

La aplicación debería utilizar los datos almacenados en `localStorage` en lugar de comenzar nuevamente desde los datos iniciales cada vez que se cambia de página o se recarga la aplicación.

Para no perderse

- ⚠ No intenten resolver todo el Hito 3 de una sola vez.

Avancen de a poco, prueben cada parte antes de continuar y utilicen el repositorio de ejemplo como referencia.

El objetivo no es copiar una solución, sino comprender los mecanismos y aplicarlos al dominio de su propio proyecto.

- 📄 Si tienen dudas sobre alguna función o sintaxis de JavaScript, consulten la documentación oficial y experimenten con ejemplos pequeños antes de incorporarla al proyecto.